GIT NOTES FOR BEGINNERS

• Concepts • Commands • Practical examples

Purpose: Learn Git instead of only memorizing commands. For each command, these notes explain what it means, why we use it, and what happens when it runs. Examples use an embedded-firmware project such as motor.c. Replace placeholders such as <branch> and <commit> with real values.

1. Basic Git concepts

Git is a distributed version control system. It records the history of source code, helps compare versions, supports parallel development, and can help recover earlier committed work.

Git

Meaning: A tool that tracks changes to files and records project history.

Why use it? To manage code changes safely and collaborate.

Repository (repo)

Meaning: A project and its Git history. A local repository is on your computer; a remote repository is hosted elsewhere.

Why use it? To keep source files and their history organized.

Working directory

Meaning: The files you currently see and edit. Edits here are not automatically committed.

Why use it? This is where you develop and modify code.

Staging area (index)

Meaning: The set of changes selected for the next commit.

Why use it? To decide exactly which changes go into a commit.

Commit

Meaning: A snapshot of staged content with a unique ID and usually a message.

Why use it? To create a checkpoint that can be reviewed later.

Branch

Meaning: A movable reference to a commit, commonly used for a feature or bug fix.

Why use it? To work separately without directly changing the main development line.

HEAD

Meaning: A reference to the current checkout, usually the current branch's latest commit.

Why use it? To understand which commit or branch your commands are operating on.

Remote

Meaning: A named connection to another repository, often called origin.

Why use it? To fetch teammates' work and share your commits.

Merge

Meaning: Integrates changes from another branch into the current branch.

Why use it? To combine completed work.

Rebase

Meaning: Replays commits on top of a different base commit.

Why use it? To update a branch and create a more linear history.

Tag

Meaning: A named reference to a commit, often used for releases.

Why use it? To mark versions such as v1.0.0.

2. Start a project and configure Git

Initialize a repository

Command: `git init`

What it means: Creates Git repository metadata in the current folder, normally in .git.

Why we use it: Use it to start tracking an existing folder with Git.

What happens: Existing files are not automatically staged or committed.

Clone a repository

Command: `git clone <url>`

What it means: Downloads a repository and its history into a new local folder.

Why we use it: Use it to get a project from GitHub or another Git server.

What happens: Git creates a working copy and normally configures the remote as origin.

Example: `git clone https://github.com/user/motor-project.git`

Clone into a named folder

Command: `git clone <url> <directory>`

What it means: Clones the repository into a folder name you choose.

Why we use it: Use it when you want to control the local directory name.

Set commit author name

Command: `git config --global user.name "Your Name"`

What it means: Sets the name recorded as the author of new commits.

Why we use it: Use it to identify your commits.

What happens: The setting applies globally unless a repository-specific setting overrides it.

Set commit author email

Command: `git config --global user.email "you@example.com"`

What it means: Sets the email recorded in new commits.

Why we use it: Use the email appropriate for your project or hosting account.

What happens: Existing commits are not changed.

Set VS Code as editor

Command: `git config --global core.editor 'code --wait'`

What it means: Configures Git to use VS Code for tasks that need a text editor.

Why we use it: Use it if you want to write commit messages in VS Code.

What happens: The --wait option tells Git to wait for the editor to close.

List configuration

Command: `git config --list`

What it means: Shows the configuration values Git is using.

Why we use it: Use it to check identity, aliases, and other settings.

List global configuration

Command: `git config --global --list`

What it means: Shows user-level configuration.

Why we use it: Use it to inspect settings that apply across repositories.

List local configuration

Command: `git config --local --list`

What it means: Shows settings for the current repository.

Why we use it: Use it to check repository-specific overrides.

Create an alias

Command: `git config --global alias.st status`

What it means: Creates a shortcut so git st runs git status.

Why we use it: Use aliases for commands you type frequently.

3. Daily workflow: status, stage, commit

Check repository status

Command: `git status`

What it means: Shows the current branch and which files are modified, staged, untracked, or conflicted.

Why we use it: Use it before and after important Git operations.

What happens: It reports state without changing your files.

Stage one file

Command: `git add <file>`

What it means: Adds the current version of a file's changes to the staging area.

Why we use it: Use it to choose exactly what should go into the next commit.

What happens: No commit is created yet.

Example: `git add motor.c`

Stage changes in the current folder

Command: `git add .`

What it means: Stages eligible changes under the current directory.

Why we use it: Use it when you want to stage changes in the folder you are working in.

What happens: Its scope depends on where you run the command.

Stage all changes

Command: `git add -A`

What it means: Stages additions, modifications, and deletions across the repository.

Why we use it: Use it when you intentionally want all current changes staged.

Stage selected portions

Command: `git add -p`

What it means: Shows change sections and lets you choose which sections to stage.

Why we use it: Use it when a file contains changes for different tasks.

What happens: Only selected sections are staged.

Commit staged changes

Command: `git commit -m "Add motor driver"`

What it means: Creates a snapshot from the staged content.

Why we use it: Use it to save one logical unit of work with a clear message.

What happens: A commit ID and commit message are added to local history; unstaged edits are excluded.

Stage tracked edits and commit

Command: `git commit -am "Fix motor timeout"`

What it means: Stages modifications and deletions of already tracked files, then commits them.

Why we use it: Use it for a quick commit when no new untracked files need to be included.

What happens: New untracked files are not included.

Move or rename a tracked file

Command: `git mv <old-path> <new-path>`

What it means: Moves or renames a file and updates Git's index.

Why we use it: Use it when reorganizing source files.

Remove a tracked file

Command: `git rm <file>`

What it means: Deletes a tracked file from the working directory and stages its deletion.

Why we use it: Use it when you want the deletion recorded in the next commit.

Stop tracking but keep local copy

Command: `git rm --cached <file>`

What it means: Removes a file from Git's index without deleting the local copy.

Why we use it: Use it when a file should no longer be tracked, often after adding it to .gitignore.

Careful: This does not remove the file from earlier commits in history.

4. Understand commits, history, graphs, and diff

Show commit history

Command: `git log`

What it means: Displays commits with details such as author, date, and message.

Why we use it: Use it to understand how the project changed.

Compact history

Command: `git log --oneline`

What it means: Shows one short line per commit.

Why we use it: Use it to scan history quickly.

Show history graph

Command: `git log --oneline --graph --all`

What it means: Displays a compact text graph of commits, branches, and merges.

Why we use it: Use it to understand where branches split and merge.

What happens: Spaces between options matter: --oneline --graph --all.

Show a commit

Command: `git show <commit>`

What it means: Shows details and usually the patch for a commit or Git object.

Why we use it: Use it to inspect what a specific commit changed.

Show unstaged changes

Command: `git diff`

What it means: Compares working-directory changes against the staging area.

Why we use it: Use it to review edits before staging.

Show staged changes

Command: `git diff --staged`

What it means: Compares staged content against the current commit.

Why we use it: Use it to review what the next commit will contain.

Show all tracked changes since HEAD

Command: `git diff HEAD`

What it means: Compares staged and working-tree tracked changes against HEAD.

Why we use it: Use it to review all tracked edits together.

Compare two commits

Command: `git diff <commit1> <commit2>`

What it means: Shows the code differences between two commits or references.

Why we use it: Use it to compare versions.

Summarize changed files

Command: `git diff --stat`

What it means: Shows a compact per-file summary of additions and deletions.

Why we use it: Use it when you need an overview rather than every changed line.

Check whitespace errors

Command: `git diff --check`

What it means: Checks changes for common whitespace problems.

Why we use it: Use it before committing to catch accidental whitespace issues.

Show a file's history

Command: `git log -- <file>`

What it means: Lists commits that changed a file.

Why we use it: Use it to investigate when a behavior or bug may have been introduced.

Follow file history through renames

Command: `git log --follow -- <file>`

What it means: Follows a file's history across detected renames.

Why we use it: Use it when a source file has been renamed.

Find commits changing a text pattern

Command: `git log -G "motor_speed" -- <file>`

What it means: Finds commits whose patches add or remove lines matching a pattern.

Why we use it: Use it to locate changes involving a variable or code expression.

Show line authorship

Command: `git blame <file>`

What it means: Shows the commit and author associated with each line's last change.

Why we use it: Use it as a starting point for code-history investigation; it does not necessarily identify the original cause of a bug.

5. Branches and branch management

A branch lets you develop one feature or bug fix separately. The branch name points to a commit; it is not a separate copy of the entire repository.

List local branches

Command: `git branch`

What it means: Lists local branches and marks the current branch with *.

Why we use it: Use it to check which branches exist.

Create a branch without switching

Command: `git branch <branch-name>`

What it means: Creates a branch at the current commit.

Why we use it: Use it to prepare a branch for later.

What happens: You remain on the current branch.

Create and switch branch

Command: `git switch -c feature/motor-control`

What it means: Creates a branch and switches to it.

Why we use it: Use it to begin a new feature or fix.

What happens: The new branch starts from the current commit.

Switch branch

Command: `git switch <branch-name>`

What it means: Changes the current branch.

Why we use it: Use it to move between tasks.

What happens: Git updates files to match the target branch if it can do so safely.

Older branch creation command

Command: `git checkout -b <branch-name>`

What it means: Creates a branch and switches to it.

Why we use it: Use it when maintaining older workflows; git switch -c is the focused modern alternative.

List remote-tracking branches

Command: `git branch -r`

What it means: Lists branches known from remote-tracking references.

Why we use it: Use it to inspect branches fetched from a remote.

List all branches

Command: `git branch -a`

What it means: Lists local branches and remote-tracking branches.

Why we use it: Use it to see both categories.

Sort branches by latest commit

Command: `git branch --sort=-committerdate`

What it means: Sorts local branches by most recent commit date.

Why we use it: Use it to find recently active branches.

Delete merged branch

Command: `git branch -d <branch>`

What it means: Deletes a local branch when Git considers it safely merged.

Why we use it: Use it after the feature is integrated.

Force-delete a branch

Command: `git branch -D <branch>`

What it means: Deletes a local branch even if unmerged work may remain.

Why we use it: Use it only when that work is intentionally no longer needed.

Careful: Commits may become difficult to find if no other reference points to them.

Rename a branch

Command: `git branch -m <new-name>`

What it means: Renames the current branch; an old and new name can also be supplied.

Why we use it: Use it to correct or standardize a branch name.

What happens: A remote branch is not automatically renamed.

6. Merge, rebase, cherry-pick, and conflicts

Merge a branch

Command:
```
git switch main
git merge feature/motor-control
```

What it means: Integrates the feature branch into the current branch.

Why we use it: Use it when you want to bring completed work into main.

What happens: Git may fast-forward or create a merge commit. Conflicts may need manual resolution.

Rebase a feature branch

Command:
```
git switch feature/motor-control
git rebase main
```

What it means: Replays feature commits on top of the latest main commit.

Why we use it: Use it to update a feature branch and keep history more linear.

What happens: Replayed commits receive new IDs.

Careful: Avoid rebasing commits teammates already use unless the team coordinates it.

Interactive rebase

Command: `git rebase -i HEAD~5`

What it means: Opens a todo list for recent commits, allowing operations such as reword, reorder, squash, or edit.

Why we use it: Use it to tidy local history before sharing.

What happens: Replace 5 with the number of commits to inspect.

Continue a rebase

Command: `git rebase --continue`

What it means: Continues a paused rebase after conflicts have been resolved and staged.

Why we use it: Use it after editing conflicted files and running git add.

Abort a rebase

Command: `git rebase --abort`

What it means: Cancels the current rebase and returns to its original state.

Why we use it: Use it if you decide not to continue.

Skip a rebased commit

Command: `git rebase --skip`

What it means: Skips the commit currently being replayed.

Why we use it: Use it only when you intentionally do not want that commit's change.

Cherry-pick a commit

Command: `git cherry-pick <commit>`

What it means: Applies one commit's changes onto the current branch, usually creating a new commit.

Why we use it: Use it to transfer a specific fix without merging an entire branch.

Merge conflict concept

Command:
```
git status
# edit conflicted files
 git add <resolved-file>
git merge --continue
```

What it means: A conflict occurs when Git cannot combine changes automatically.

Why we use it: Use status to find conflicted files, edit them to the correct result, stage the resolved files, then continue the operation.

What happens: For a merge, use git merge --continue if Git requests it; for a rebase, use git rebase --continue. You can often cancel with the relevant --abort command.

7. Remote repositories: fetch, pull, and push

List remote URLs

Command: `git remote -v`

What it means: Shows remote names and fetch/push URLs.

Why we use it: Use it to confirm where the repository connects.

Add a remote

Command: `git remote add origin <url>`

What it means: Registers a remote URL under a name such as origin.

Why we use it: Use it to connect a local repository to a hosted repository.

What happens: It does not upload or download commits by itself.

Remove a remote

Command: `git remote remove <name>`

What it means: Removes the remote configuration from your local repository.

Why we use it: Use it when a connection is no longer needed.

What happens: It does not delete the hosted repository.

Fetch remote updates

Command: `git fetch origin`

What it means: Downloads commits and updates remote-tracking references without integrating into the current branch.

Why we use it: Use it to inspect remote changes before merging or rebasing.

Pull remote changes

Command: `git pull`

What it means: Fetches and integrates updates into the current branch according to the configured strategy.

Why we use it: Use it to bring your local branch up to date.

What happens: The integration may use merge or rebase depending on configuration.

Pull with rebase

Command: `git pull --rebase`

What it means: Fetches remote changes and replays local commits on top of the updated upstream.

Why we use it: Use it when a linear history is preferred.

Push a branch

Command: `git push origin <branch>`

What it means: Uploads commits from a local branch to a remote branch.

Why we use it: Use it to share your committed work.

What happens: The push can be rejected by permissions, branch protection, or non-fast-forward history.

First push and set upstream

Command: `git push -u origin <branch>`

What it means: Pushes the branch and records its upstream tracking branch.

Why we use it: Use it the first time you publish a new branch.

Force push with a safety check

Command: `git push --force-with-lease`

What it means: Force-updates remote history only if the remote state matches what Git expects.

Why we use it: Use it only when rewriting remote history is intentional.

Careful: It can still overwrite shared history; coordinate with teammates.

8. Stash: temporarily save unfinished work

Stash is temporary storage for selected uncommitted changes, not a permanent backup. It is useful when you need to switch tasks before your code is ready to commit.

Stash changes

Command: `git stash`

What it means: Saves tracked staged and unstaged changes and cleans them from the working tree.

Why we use it: Use it to switch tasks without making an unfinished commit.

Stash with a message

Command: `git stash push -m "motor work in progress"`

What it means: Saves changes with a descriptive label.

Why we use it: Use it when you have several stashes.

Include untracked files

Command: `git stash -u`

What it means: Includes untracked files along with tracked changes.

Why we use it: Use it when new files also need to be put aside.

List stashes

Command: `git stash list`

What it means: Lists saved stash entries.

Why we use it: Use it to find saved work.

Show stash patch

Command: `git stash show -p`

What it means: Displays the detailed changes stored in a stash.

Why we use it: Use it to inspect saved work before restoring.

Apply and remove latest stash

Command: `git stash pop`

What it means: Applies the latest stash and removes it if application succeeds.

Why we use it: Use it to resume work.

Apply but keep stash

Command: `git stash apply`

What it means: Applies the latest stash without removing the entry.

Why we use it: Use it when you may need the stash again.

Delete one stash

Command: `git stash drop stash@{0}`

What it means: Removes a selected stash entry.

Why we use it: Use it when the saved changes are no longer needed.

Delete all stashes

Command: `git stash clear`

What it means: Removes every stash entry.

Why we use it: Use it only when you intentionally want to discard all stashed work.

9. Undo and recovery

Choose an undo command based on what you want to change: working files, staging area, branch position, or shared history. Run `git status` first.

Discard unstaged edits in one file

Command: `git restore <file>`

What it means: Restores a file from the index by default.

Why we use it: Use it when you intentionally want to discard unstaged edits.

Careful: The discarded edits may be difficult to recover.

Unstage a file

Command: `git restore --staged <file>`

What it means: Removes a file from staging while keeping its working-tree edits.

Why we use it: Use it when you staged the wrong file.

Unstage using reset

Command: `git reset <file>`

What it means: Unstages the selected file while preserving working-tree edits.

Why we use it: Use it as an alternative to git restore --staged.

Soft reset

Command: `git reset --soft HEAD~1`

What it means: Moves HEAD back one commit while keeping changes staged.

Why we use it: Use it to redo a local commit without losing its staged content.

Mixed reset

Command: `git reset --mixed HEAD~1`

What it means: Moves HEAD and resets the staging area while leaving working files changed.

Why we use it: Use it to undo a commit but leave edits unstaged.

Hard reset

Command: `git reset --hard <commit>`

What it means: Moves HEAD and resets the index and tracked working files to the selected commit.

Why we use it: Use it only when you intentionally want to discard tracked changes.

Careful: This can permanently discard uncommitted tracked work.

Revert a commit

Command: `git revert <commit>`

What it means: Creates a new commit that reverses the changes from an earlier commit.

Why we use it: Use it to undo shared history without rewriting existing commits.

Revert latest commit

Command: `git revert HEAD`

What it means: Creates a new commit that reverses the latest commit.

Why we use it: Use it when a committed change needs to be undone transparently.

Inspect recent reference movements

Command: `git reflog`

What it means: Shows recent movements of HEAD and other references.

Why we use it: Use it to locate an earlier commit after a reset, rebase, or branch movement.

What happens: Entries expire and are not a permanent backup.

Create a recovery branch

Command: `git branch recovery <commit-id>`

What it means: Creates a new branch pointing to a chosen commit.

Why we use it: Use it after locating a likely lost commit in reflog, so you can inspect it safely.

Preview untracked cleanup

Command: `git clean -n`

What it means: Shows untracked files that would be removed.

Why we use it: Use it before cleanup to preview the effect.

Remove untracked files

Command: `git clean -f`

What it means: Deletes untracked files.

Why we use it: Use it only after reviewing the preview.

Careful: This is destructive; git clean -fd can also remove untracked directories.

Amend latest commit

Command: `git commit --amend -m "Correct message"`

What it means: Replaces the most recent commit, often to fix its message or include staged changes you forgot.

Why we use it: Use it before sharing the commit when a correction is needed.

Careful: The commit ID changes. Avoid amending shared commits.

10. Tags and release versions

List tags

Command: `git tag`

What it means: Lists local tags.

Why we use it: Use it to see version labels.

Create lightweight tag

Command: `git tag v1.0.0`

What it means: Creates a simple tag pointing to the current commit.

Why we use it: Use it to mark a version or milestone.

Create annotated tag

Command: `git tag -a v1.0.0 -m "Release 1.0.0"`

What it means: Creates a tag with a message and tagger metadata.

Why we use it: Use it for release versions.

Show tag details

Command: `git show v1.0.0`

What it means: Shows the commit and details associated with the tag.

Why we use it: Use it to verify what the release label points to.

Push one tag

Command: `git push origin v1.0.0`

What it means: Uploads one tag to the remote.

Why we use it: Use it to publish a specific version.

Push all tags

Command: `git push origin --tags`

What it means: Uploads all local tags.

Why we use it: Use it when you intend to publish all tags.

11. Submodules

A submodule is a separate Git repository used inside another repository. The parent project records the specific commit expected for that dependency.

Add a submodule

Command: `git submodule add <url> <path>`

What it means: Adds another repository at a path inside the current project.

Why we use it: Use it when a dependency is maintained in its own repository.

Initialize submodules

Command: `git submodule init`

What it means: Initializes local configuration for submodules already recorded by the project.

Why we use it: Use it when setting up a project containing submodules.

Update submodules

Command: `git submodule update`

What it means: Checks out the submodule commits recorded by the parent repository.

Why we use it: Use it to get the dependency versions expected by the project.

Initialize and update recursively

Command: `git submodule update --init --recursive`

What it means: Initializes and checks out submodules, including nested submodules.

Why we use it: Use it after cloning a project that contains submodules.

Run a command in every submodule

Command: `git submodule foreach 'git pull origin main'`

What it means: Runs a command inside each checked-out submodule.

Why we use it: Use it only when you intentionally want to update each submodule to its remote main branch.

Careful: This can move submodules away from the commits recorded by the parent repository.

12. Git hooks, .gitignore, .env, and shell echo

These concepts are useful in software projects but are not all Git commands.

Git hooks

Hooks are scripts that can run around Git operations, such as pre-commit or pre-push. Teams use them to run formatting, tests, or commit-message checks. Local hooks need to be installed/configured; they are not automatically shared simply by cloning a repository.

.gitignore

```
.env
build/
*.log
```

A .gitignore file lists patterns for untracked files Git should normally ignore, such as build output and local configuration. It does not automatically stop tracking files already committed.

.env files

```
DEBUG=true
SERVER_PORT=8080
```

An .env file is a common way to store local key-value configuration. The application or a library must load it before the values are available. Never commit real passwords, API keys, or other secrets.

Shell echo and redirection

```
echo "Hello"
echo "DEBUG=true" > .env
echo "LOG_LEVEL=info" >> .env
```

echo prints text in the shell. The > operator writes output and overwrites the target file; >> appends to the file. echo is a shell command, not a Git command.

13. Git options and hyphen flags

A short option usually begins with one hyphen, such as -m. A long option usually begins with two, such as --staged. The meaning depends on the Git command.

- **-m** — In git commit, supplies a commit message: git commit -m "Fix bug".
- **-c** — In git switch, creates and switches to a branch: git switch -c feature.
- **-b** — In git checkout, creates and switches to a branch: git checkout -b feature.
- **-d / -D** — For git branch, -d safely deletes a merged branch; -D forces deletion.
- **-u** — For git push, sets upstream tracking: git push -u origin feature.
- **-a** — Meaning varies by command; git commit -a includes tracked modifications and deletions.
- **-p** — Meaning varies: git add -p stages selected hunks; git log -p shows patches.
- **--staged** — For git diff, displays staged changes.
- **--all** — For git log, includes commits reachable from all refs; in other commands its meaning may differ.
- **--oneline** — For git log, displays a compact one-line entry per commit.
- **--graph** — For git log, draws a text graph of commit history.
- **--soft / --mixed / --hard** — Reset modes that determine what happens to the branch, staging area, and tracked working files.
- **--continue / --abort / --skip** — Options used in operations such as rebase to continue, cancel, or skip a step.
- **--force-with-lease** — A guarded force-push option; still use carefully on shared branches.

14. Common workflow for embedded firmware

Example: implement motor timeout handling in an embedded C project. This assumes the repository and target branch exist.

```
git status
```
Check the current branch and local changes.

```
git switch -c feature/motor-timeout
```
Create a separate branch for the task.

```
# Edit motor.c and motor.h
```
Implement and test the feature.

```
git diff
```
Review unstaged changes.

```
git add motor.c motor.h
```
Stage the intended files.

```
git diff --staged
```
Review exactly what will be committed.

```
git commit -m "Add motor timeout handling"
```
Save the change in local history.

```
git fetch origin
```
Download remote updates for inspection.

```
git push -u origin feature/motor-timeout
```
Publish the feature branch.

```
# Open a pull request and merge after review
```
Follow the team's review and integration process.

15. Troubleshooting and safety rules

- Run git status before and after important operations.
- Review git diff before staging and git diff --staged before committing.
- git commit -am does not include new untracked files.
- git fetch downloads updates but does not integrate them into your current branch; git pull fetches and integrates.
- git restore <file> discards unstaged edits; git restore --staged <file> unstages while preserving working edits.
- git reset --hard and git clean -f are destructive. Preview and back up important work first.
- Rebase changes commit IDs. Avoid rewriting commits teammates already use.
- git revert is often safer for undoing a commit that has already been shared.
- Stash is temporary storage, not a permanent backup.
- A .gitignore entry does not untrack a file that Git already tracks.
- Do not commit secrets from .env files.

16. Quick revision table

Command	Remember it as
git init	Start a repository.
git clone <url>	Download a repository.
git status	Check current state.
git add <file>	Stage selected changes.
git commit -m "message"	Record a snapshot.
git log --oneline --graph --all	View compact commit graph.
git diff	Review unstaged changes.
git diff --staged	Review staged changes.
git switch -c <name>	Create and switch branch.
git merge <branch>	Integrate branch changes.

git rebase <branch>	Replay commits on a new base.
git stash	Temporarily put tracked changes aside.
git reflog	Inspect recent reference movements.
git fetch	Download remote updates without integrating.
git pull	Fetch and integrate remote changes.
git push	Upload commits.
git revert <commit>	Undo by creating a reversing commit.
git restore <file>	Discard unstaged edits in a file.
git cherry-pick <commit>	Apply a selected commit.
git tag v1.0.0	Mark a version.
git help <command>	Read command documentation.

17. Notes source and scope

This revised guide incorporates the concepts and commands from the supplied Markdown notes, the Git cheat-sheet material shared in the conversation, and additional beginner explanations for hooks, .gitignore, .env, shell echo, aliases, conflict handling, flags, safety, and embedded-firmware workflow. The source Markdown included a quick revision table and a motor-project workflow; these have been retained and expanded. Examples are instructional and should be adapted to the actual repository.

18. Command flags, options, arguments, and common words

A Git command often has four parts: the program (git), the command (commit), options/flags (-m), and values/arguments ("Fix motor timeout"). Not every option is a flag, and not every word after a command is an option.

Example: git commit -m "Fix motor timeout"

Part	Example	What it means	Important detail
Program	git	Runs Git.	Usually the first word.
Command	commit	Tells Git what operation to do.	Other commands include status, add, log, push.
Option / flag	-m	Changes how the command behaves or supplies a mode.	Options can be short (-m) or long (--message).
Argument / value	"Fix motor timeout"	The value used by the command or option.	Here it is the commit message.

Common options and terms you will see

The exact number of Git flags is not a fixed small number. Git has many commands, and each command has its own options; new Git versions can add options. Learn the common ones by command rather than trying to memorize one total.

Option / term	Used with	Meaning	Example / note
-m	git commit; git tag -a	Message option: supplies a message.	git commit -m "Fix sensor timeout"
-b	git switch -c; git checkout -b	In these commands, create a new branch.	git switch -c feature/motor-stop. -b is not a universal Git flag.
-c	git switch; git branch; git checkout	Usually means create a branch in these specific commands.	git switch -c feature/logging
-C	git switch; git checkout	In supported commands, create/reset a branch to a start point.	Capitalization matters; check help before use.
-a	git commit; git branch; other commands	Meaning depends on the command. For git commit -a, automatically stage tracked modified/deleted files before committing; it does not include new untracked files.	git commit -am 'Fix bug'
-p	git add; git reset; git checkout/restore variants	Patch/interactive mode: choose changes in pieces, depending on command.	git add -p
-f	git push; git branch; other commands	Force/override behavior in some commands; exact effect depends on command.	Dangerous if used without understanding.
--global	git config	Apply a setting for your user across repositories.	git config --global user.name "Your Name"
--local	git config	Apply a setting to the current repository.	Local config commonly overrides global config.
--system	git config	Apply configuration for all users on the system, if permitted.	Usually requires administrator rights.
--oneline	git log	Compact one-line commit history.	git log --oneline --graph --all
--graph	git log	Draw branch/merge relationships using text characters.	Useful with --oneline --all.
--all	git log; git branch; others	Include all refs or all branches, according to the command.	Meaning is command-specific.
--staged	git diff; git restore variants	Refer to staged/index changes in commands that support it.	git diff --staged

--cached	git diff; git rm	Often refers to the staging index; exact meaning depends on command.	git diff --cached
--amend	git commit	Replace the most recent commit with a new commit.	Avoid amending a commit already shared unless the team agrees.
--no-ff	git merge	Create a merge commit even when fast-forward is possible.	Can preserve an explicit merge point.
--ff-only	git merge; git pull configurations	Only allow a fast-forward; fail if a merge commit would be needed.	Useful when you want to avoid automatic merge commits.
--rebase	git pull; git pull configuration	Integrate changes by rebasing instead of merging, when supported.	Know whether your team expects merge or rebase.
--force-with-lease	git push	Force-push only if the remote ref is still at the value you expect.	Safer than --force, but still rewrites shared history.
-u / --set-upstream	git push	Set upstream tracking for the branch.	git push -u origin feature/motor-stop
--delete / -d	git branch; git push	Delete a branch, with exact local/remote syntax depending on command.	git branch -d feature/motor-stop
--hard	git reset	Move HEAD/index/working files to the target commit.	Can discard uncommitted changes; use with care.
--soft	git reset	Move HEAD but keep changes staged.	Useful for rebuilding the last commit.
--mixed	git reset	Move HEAD and reset staging, while keeping working-file edits.	Default mode of git reset.
--	git add; git restore; git checkout; others	Separates options from file paths.	git restore -- motor.c
HEAD	Many commands	Reference to current checkout/commit.	HEAD~1 means one commit before HEAD in common cases.
origin	Remote-related commands	A conventional remote nickname, often created by clone.	Not a reserved keyword; it can be renamed.
main	Branch-related commands	A common default branch name.	Not a special Git command or guaranteed branch name.
upstream	Branch/remote context	Can mean a branch's tracking branch, or a remote named upstream in fork workflows.	Meaning depends on context.
URL	clone; remote add	Address of a remote repository.	HTTPS and SSH URLs are common.
<file>	Many commands	Placeholder meaning: replace it with a real file path.	Do not type the angle brackets literally.

How to read command examples

- Angle brackets such as <branch>, <commit>, <file>, and <url> are placeholders. Replace them with real values; normally do not type the brackets.
- Quotes protect a message or path containing spaces: git commit -m "Fix motor stop timeout".
- A slash in a branch name is allowed and often used to organize names: feature/motor-stop.
- Capital letters can change meaning: -f and -F may do different things. Git options are command-specific.
- A hyphenated long option usually starts with two hyphens: --global. A short option often starts with one: -m.
- Some options can be combined, but do not assume all can. Prefer clear commands while learning.

To see exact options for the installed Git version, use git help <command> or git <command> -h. For example: git help commit and git switch -h. The short -h help form is supported by most Git commands, but git help is the dependable general approach.

19. Practical command flags: worked examples

Commit with a message

Syntax: git commit -m "Fix motor timeout"

Meaning: -m passes the commit message to Git.

Why/when: Use it to describe the purpose of the commit.

What happens: Git records the currently staged changes as a commit with that message.

Example: git add motor.c && git commit -m "Fix motor timeout"

Caution: Only staged changes are included. -m does not stage files.

Create and switch to a branch

Syntax: git switch -c feature/motor-stop

Meaning: -c tells git switch to create the branch and switch to it.

Why/when: Use it when starting a feature or bug fix.

What happens: Git creates the branch at the current commit and makes it the current branch.

Example: git switch -c fix/propel-motor

First push and set tracking

Syntax: git push -u origin feature/motor-stop

Meaning: -u is short for --set-upstream.

Why/when: Use it for a new local branch that does not yet track a remote branch.

What happens: Git pushes the branch to the remote named origin and records the upstream relationship if successful.

Example: git push -u origin fix/propel-motor

Caution: origin must be configured and you must have permission to push.

Show compact branch history

Syntax: git log --oneline --graph --decorate --all

Meaning: --oneline shortens each commit; --graph draws relationships; --decorate shows names like branches/tags; --all includes refs from all branches.

Why/when: Use it to understand branch and merge history quickly.

What happens: Git prints a compact graph in the terminal.

Example: git log --oneline --graph --decorate --all

Show only staged changes

Syntax: git diff --staged

Meaning: --staged asks Git to compare the staging area with the last commit.

Why/when: Use it before committing to review exactly what will be committed.

What happens: Git displays staged differences.

Example: git add motor.c && git diff --staged

Restore one file from the last commit

Syntax: git restore --source=HEAD -- motor.c

Meaning: --source=HEAD selects the version from HEAD; -- separates options from the file path.

Why/when: Use it only when you deliberately want to discard current unstaged edits to that file.

What happens: The working copy of motor.c is replaced with the version from HEAD.

Example: git restore --source=HEAD -- motor.c

Caution: This can destroy uncommitted edits in that file. Review git diff first.

Force-push with a safety check

Syntax: git push --force-with-lease

Meaning: Requests a history rewrite but checks that the remote ref has not changed unexpectedly.

Why/when: Sometimes needed after rewriting your own branch with rebase.

What happens: The push can still replace remote history if the lease check passes.

Example: git push --force-with-lease origin feature/motor-stop

Caution: Coordinate with teammates; never use force-push casually on shared branches.

20. More important real-world concepts

Detached HEAD

Syntax: git switch --detach <commit>

Meaning: Checks out a commit directly instead of placing HEAD on a branch.

Why/when: Useful for inspecting or testing an old version without switching a named branch.

What happens: You can inspect/build code, but new commits may not belong to a branch; create a branch if you want to keep them.

Example: git switch --detach a1b2c3d

Find which commit introduced a bug

Syntax: git bisect start; git bisect bad; git bisect good <known-good-commit>

Meaning: Bisect performs a binary search through commit history using good/bad test results.

Why/when: Use it when a bug appeared sometime across many commits.

What happens: Git checks out candidate commits; test each one and report good/bad until Git narrows the first bad commit.

Example: git bisect reset when finished

Caution: Always run git bisect reset when finished to return to the previous checkout.

Search tracked source files

Syntax: git grep "Motor_Init"

Meaning: Searches tracked files in the repository for matching text.

Why/when: Useful for finding function names, error strings, and configuration references.

What happens: Git prints matching file paths and lines.

Example: git grep "Motor_Init"

Work on two branches in separate folders

Syntax: git worktree add ../motor-hotfix hotfix/motor

Meaning: Creates another working directory attached to a branch or commit.

Why/when: Useful when you need to inspect or fix another branch without repeatedly switching your current folder.

What happens: Git creates the additional working tree if the branch/path setup is valid.

Example: git worktree list

Caution: A branch normally cannot be checked out in two linked worktrees at once.

Inspect branch tracking

Syntax: git branch -vv

Meaning: Shows local branches and their upstream tracking branch, when configured.

Why/when: Use it to diagnose whether your local branch tracks origin/feature-name and whether it is ahead/behind.

What happens: Git prints branch names, commit summaries, and tracking information.

Example: git branch -vv

Show remotes and their URLs

Syntax: git remote -v

Meaning: Lists remote names and fetch/push URLs.

Why/when: Use it to verify where fetch and push operations will go.

What happens: Git prints configured remote addresses.

Example: git remote -v

Caution: Check the URL before pushing private or company code.

21. Merge conflicts: complete safe workflow

A conflict means Git cannot automatically combine overlapping changes. It is common when two branches edit the same lines. A conflict is not a sign that the repository is broken.

1. Run git status to see the operation and conflicted files.
2. Open each conflicted file. Git may show markers like <<<<<<<, =======, and >>>>>>>.
3. Decide the correct final content. Keep the right code from one side or combine both; remove all conflict markers.
4. Run the relevant tests or build. For firmware, compile the project and check the affected behavior when possible.
5. For a merge, stage resolved files with git add <file>, then run git commit if Git has not already completed the merge.
6. For a rebase, stage resolved files with git add <file>, then run git rebase --continue.

- Cancel a merge with git merge --abort if you want to return to the pre-merge state and Git can do so safely.
- Cancel a rebase with git rebase --abort if you want to return to the pre-rebase state.
- Do not blindly choose 'ours' or 'theirs' for firmware code: it can silently remove a required fix.

Memory aid: STATUS → EDIT → TEST → ADD → CONTINUE/COMMIT.

22. Team workflow: branch to Pull Request

7. Update your base branch: git switch main, then git pull --ff-only (use the team-approved integration method).
8. Create a task branch: git switch -c feature/motor-stop.
9. Make a small, focused change and inspect it with git diff.
10. Stage and commit: git add motor.c; git diff --staged; git commit -m "Add motor stop handling".
11. Push the branch: git push -u origin feature/motor-stop.
12. Open a Pull Request (PR) on GitHub/GitLab/another hosting platform. Explain the problem, the change, and the tests.
13. Address review comments with new commits or an agreed history cleanup; rerun tests.
14. After approval, merge according to team policy and clean up the branch when safe.

Git itself does not create a Pull Request. A hosting platform adds review, approvals, checks, and merge buttons around Git repositories.

23. Git recovery: choose the right tool

Situation	Typical tool	What it changes	Caution
Unstaged edits to a file should be discarded	git restore -- <file>	Working-tree file	May permanently discard those edits.
File is staged but should be unstaged	git restore --staged -- <file>	Staging area only	Working-file edits remain.
Undo a published commit safely	git revert <commit>	Creates a new commit that reverses the change	Resolve conflicts if they occur.
Move HEAD back but keep	git reset --soft <commit>	Branch pointer; staging	Changes history; be careful if

changes staged		retained	commits were pushed.
Move HEAD back and unstage changes	git reset <commit>	Branch pointer and index; working edits generally remain	Check status before proceeding.
Reset working files to a commit	git reset --hard <commit>	Branch pointer, index, and tracked working files	Destructive; uncommitted changes can be lost.
Find a previous local HEAD/branch position	git reflog	Reads local reference movement history	Entries expire; not a guaranteed permanent backup.
Undo a rebase while it is still running	git rebase --abort	Returns to pre-rebase state when possible	Use before trying unrelated recovery commands.
Accidentally deleted a local branch	Inspect git reflog, then create a branch at the recovered commit	Creates a new branch pointer	Act promptly; identify the correct commit first.

24. Firmware repository hygiene

- .gitignore: list files Git should normally ignore, such as build output, temporary files, and local IDE state. Already-tracked files remain tracked until removed from the index.
- .gitattributes: set repository rules such as line endings and how certain file types are treated.
- .env and secrets: never commit passwords, access tokens, private keys, or production credentials. If a secret was pushed, removing it in a later commit does not make the leaked secret safe; rotate/revoke it and follow your organization's cleanup process.
- Generated files: decide whether generated binaries, map files, logs, and build output should be committed. Usually source/configuration is tracked; build output is often regenerated, but project rules vary.
- Binary firmware files: Git can store binaries, but diffs are not as useful as source diffs. Consider Git LFS or a release/artifact system for large files, according to team policy.
- Line endings: Windows commonly uses CRLF and Linux/macOS commonly use LF. Configure consistent repository rules to avoid noisy diffs.
- Before a release: build from a clean commit, run tests, review the diff, tag the tested commit (for example v1.2.0), and record relevant firmware/hardware compatibility notes.

25. Troubleshooting quick reference

Problem	First check	Likely next step
I do not know what changed	git status; git diff; git diff --staged	Review unstaged and staged changes separately.
I committed the wrong files	git show --stat HEAD; git status	If not shared, consider git commit --amend; otherwise coordinate before rewriting history.
Push rejected because remote is ahead	git fetch; git status; git log --oneline --graph --all	Integrate remote changes with the team's merge/rebase policy, then push.
My branch has no upstream	git branch -vv	For a new branch, use git push -u origin <branch> if correct.
I am in the middle of a conflict	git status	Resolve files, stage them, then continue or abort the current operation.
I do not know a command's options	git help <command>; git <command> -h	Read the installed Git version's help before trying destructive options.
A file is ignored unexpectedly	git check-ignore -v <file>	Find which ignore rule matches; do not force-add secrets.
I cannot find a commit	git log --all --oneline --graph; git reflog	Identify the commit before creating a recovery branch.

26. Quick recall: the daily Git sequence

Before work: git status → git switch <branch> → git pull (use your team's policy).

During work: edit → git diff → git add <file> → git diff --staged → git commit -m "Clear message".

Sharing: git push -u origin <branch> for a new branch; later, usually git push.

When unsure: git status first. Then use git diff, git log, git show, and git help <command> to inspect before changing anything.

Golden rule: inspect first, make small changes, test, commit clear checkpoints, and be careful with reset --hard, clean, and force-push.

27. Review notes and accuracy scope

This document is a practical learning reference, not an exhaustive list of every Git option. Git has many flags and each command has its own option set; exact support can vary by installed Git version. Use git help <command> to confirm syntax on your machine.

Terms such as origin, main, HEAD, and upstream are names/references with context-dependent meaning, not interchangeable flags. -m and -b are options whose meaning depends on the command. Always read a command as a whole rather than memorizing a flag in isolation.

Review note: commands that discard data or rewrite published history are labeled with cautions. If you are working in a company repository, follow the team's branch, merge, release, and security policies.